



Arquitetura e Design de Sistemas com IA

MÓDULO 1 SEMANA 05 - AULA 01



SCITI . SC

AGENDA

- Contextualização e definição do problema
- Arquitetura do sistema
- Diagramas C4 e UML
- UX, mockups e protótipos

OBJETIVO

Ao final da aula você será capaz de:

- Definir a arquitetura de um sistema com IA
- Criar um fluxo completo (IA + backend)
- Gerar mockups, fluxos e diagramas com IA

Cronograma

Semana 5

- **Segunda (04/05):** Modelagem de arquitetura e UX com IA
- **Terça (05/05):** Estruturação de requisitos e backlog com IA
- **Quinta (07/05):** Organização de código e fluxo de desenvolvimento com IA

Mini Projeto

- Apresentação oral (14/05)
- Entrega do mini-projeto (22/05)

Definição do problema



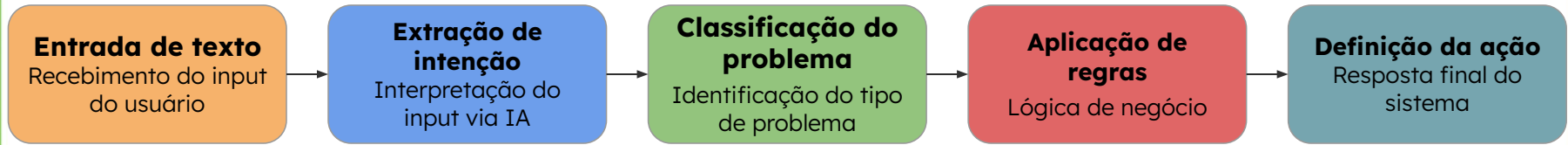
Papel da IA no sistema



IA: interpretação de linguagem natural

Sistema: regras e decisões estruturadas

Decomposição do sistema



SDLC tradicional vs desenvolvimento assistido por IA

Tradicional	IA-Assisted
• Definição de requisitos • Design de arquitetura • Implementação • Testes • Deploy e operação	• Requisitos estruturados com IA • Arquitetura assistida e avaliada • Implementação assistida e refatoração • Geração e ampliação de testes • Deploy automatizado e operação (CI/CD)

Todas as etapas com validação humana

Atuação da IA no ciclo de desenvolvimento (SDLC)

Requisitos

- Geração de user stories
- Definição de critérios de aceitação
- Identificação de ambiguidades

Arquitetura

- Geração de propostas arquiteturais
- Criação de diagramas (UML/C4)
- Avaliação de trade-offs

Implementação

- Geração de código base
- Refatoração assistida
- Análise de erros

Testes

- Geração de casos de teste
- Criação de mocks
- Identificação de edge case (casos de borda)

IA como copiloto no desenvolvimento

Atuação da IA	Papel do desenvolvedor
• Geração de artefatos intermediários (código, diagramas, texto) • Apoio na exploração de soluções e alternativas • Processamento semântico de entradas não estruturadas • Aceleração de tarefas repetitivas e operacionais	• Definição de arquitetura e decisões estruturais • Validação técnica e coerência do sistema • Garantia de consistência entre componentes e camadas • Responsabilidade sobre qualidade, comportamento e confiabilidade

Limitações da IA no desenvolvimento de sistemas

Contexto limitado

- **Ausência** de visão global do sistema
- **Dificuldade** em manter contexto entre interações
- **Dependência** de contexto explícito no prompt

Consistência sistêmica

- **Inconsistência** entre componentes gerados
- **Falta** de alinhamento com decisões arquiteturais
- **Dificuldade** em manter padrões ao longo do sistema

Tomada de decisão

- **Incapacidade** de avaliar trade-offs complexos
- **Ausência** de responsabilidade sobre decisões
- **Tendência** a respostas plausíveis, não necessariamente corretas

Qualidade e confiabilidade

- **Geração** não determinística de código e respostas
- **Erros** silenciosos difíceis de detectar
- **Necessidade** de validação e revisão constante

IA não garante coerência sistêmica sem validação

O que é arquitetura de software

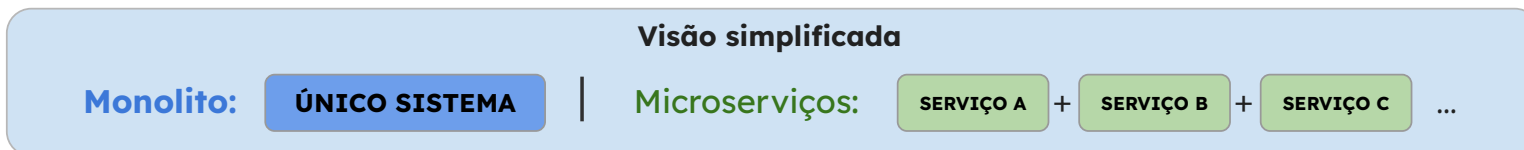
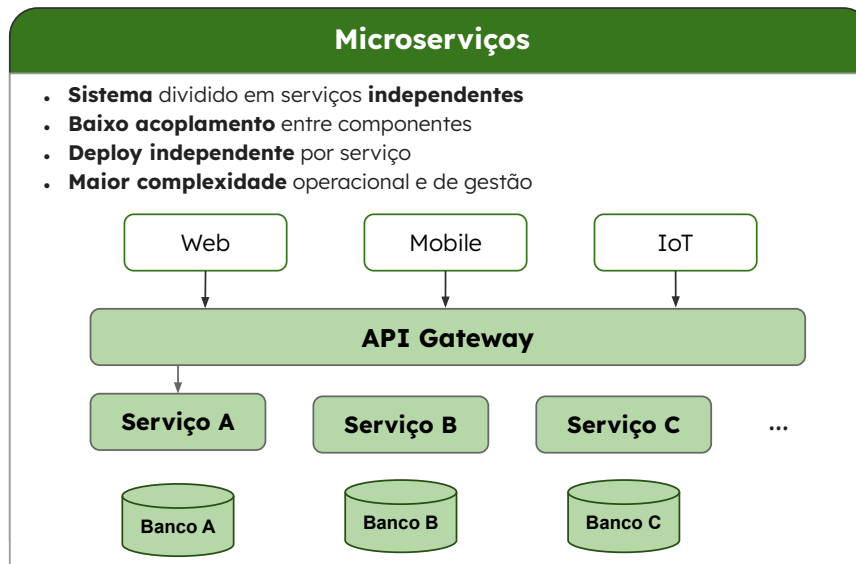
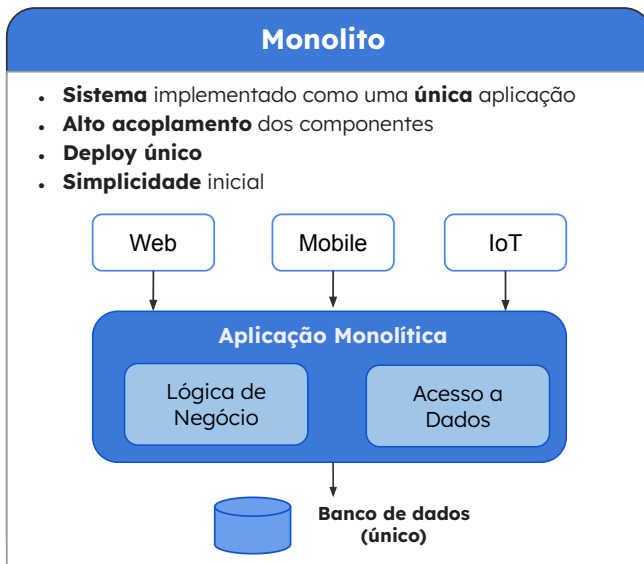
Arquitetura define:

- **Definição** de componentes e suas responsabilidades
- **Estruturação** das interações entre componentes
- **Organização** do fluxo de dados e do controle de execução
- **Separação** de responsabilidades ao longo do sistema

Arquitetura responde:

- Onde cada decisão é tomada no sistema
- Como o sistema escala e evolui
- Como manter consistência ao longo do tempo

Tipos de arquitetura de software



A escolha da arquitetura impacta diretamente a evolução e a complexidade do sistema.

Arquitetura full-stack

Frontend

- Interface com o usuário
- Captura e exibição de dados

Backend

- Orquestração e regras de negócio
- Integração entre componentes

Serviços / APIs

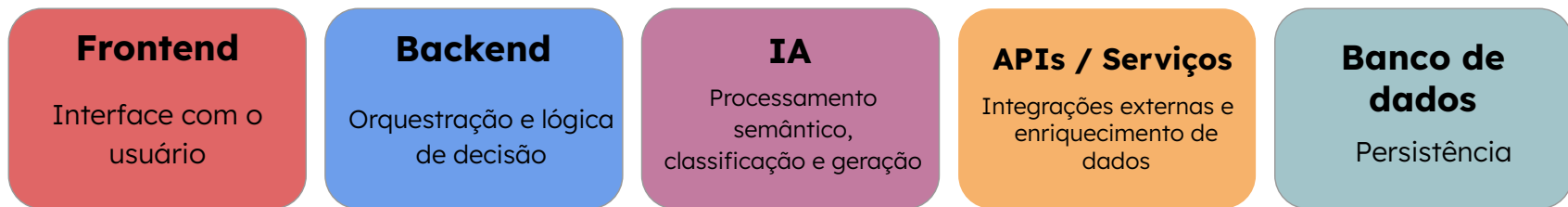
- Integrações externas
- Processamento especializado e integração externa

Banco de dados

- Persistência e recuperação de dados

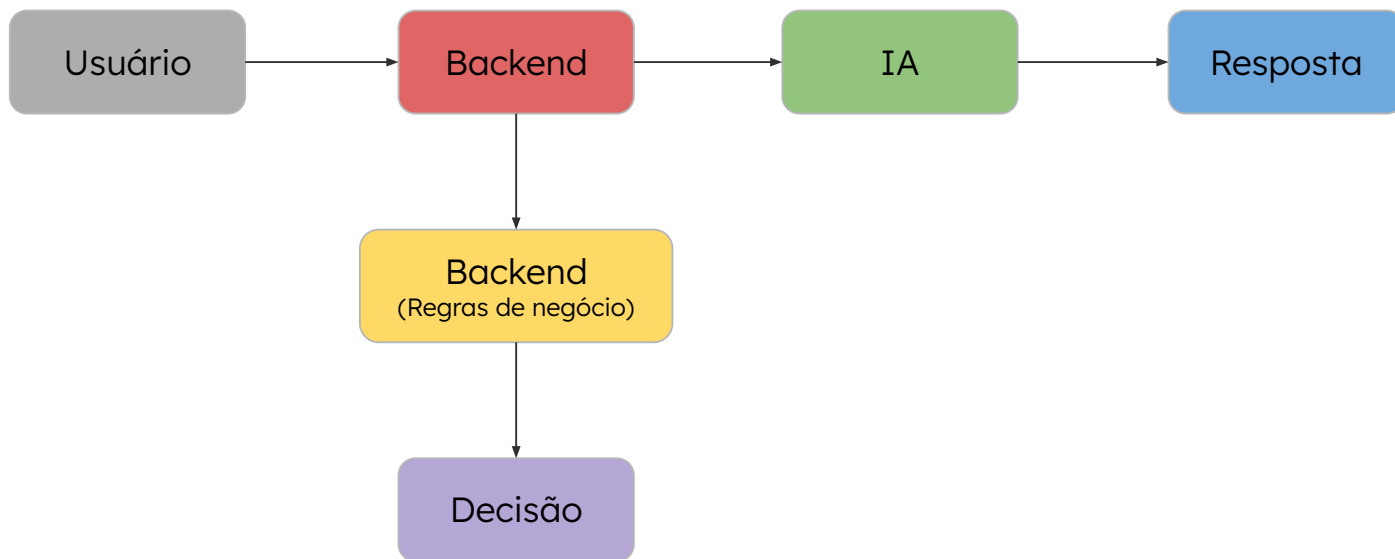
A separação em camadas reduz acoplamento e facilita evolução

Arquitetura de um sistema com IA



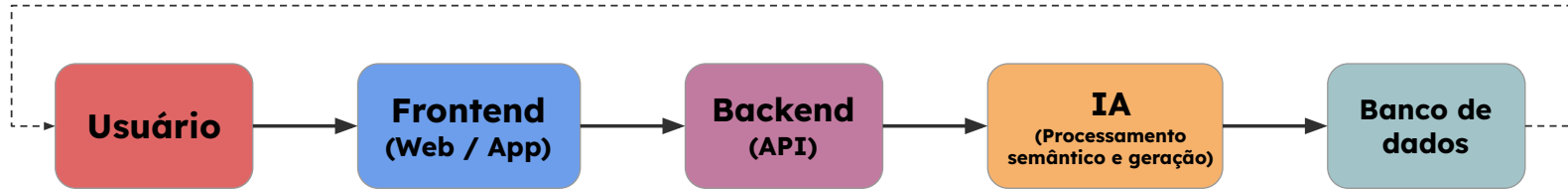
IA é um componente do sistema, não substitui a arquitetura nem a orquestração

Orquestração do backend com IA



A IA interpreta ou gera informação, mas quem controla o fluxo, aplica regras, chama serviços e decide a resposta final é o backend.

Exemplo de arquitetura para um sistema com IA



Resposta do sistema retorna ao usuário

A arquitetura define o fluxo de execução do sistema

IA como componente arquitetural vs IA como ferramenta de engenharia

IA como componente do sistema

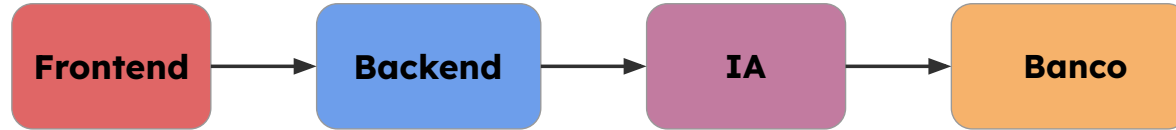
- Faz parte da solução
- Interpreta, classifica ou gera informação
- É chamada pelo backend ou por outro serviço

IA como apoio ao arquiteto/desenvolvedor

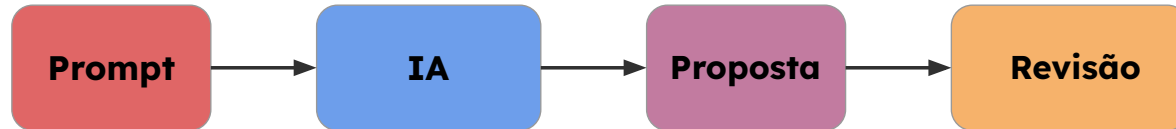
- Ajuda a propor arquiteturas
- Gera diagramas e descrições
- Apoia análise crítica e refinamento

IA como componente arquitetural vs IA como ferramenta de engenharia

IA como componente arquitetural



IA no processo de design



Prompt para geração de arquitetura com IA

Você é um arquiteto de software.

Projete a arquitetura de um sistema com as seguintes características:

- Tipo de aplicação: sistema de suporte ao usuário
- Interface: web e mobile
- Backend: API para processamento e orquestração
- IA: classificar problemas e sugerir ações
- Banco de dados: armazenamento de interações

Descreva a arquitetura no nível de containers (C4), incluindo:

- principais componentes
- responsabilidades
- fluxo de interação entre eles

Apresente a resposta de forma estruturada, descrevendo cada container e suas responsabilidades.

Análise crítica da arquitetura gerada por IA

Estrutura e separação de responsabilidades:

- Os componentes estão bem definidos?
- Existe separação clara entre responsabilidades?
- Há acoplamento excessivo entre partes?

Coerência arquitetural:

- A arquitetura segue um padrão consistente?
- Os componentes fazem sentido dentro do contexto do sistema?
- Existe alinhamento entre frontend, backend e IA?

Fluxo de execução:

- O fluxo entre os componentes está claro?
- A orquestração está bem definida?
- A IA está corretamente posicionada no fluxo?

Escalabilidade e evolução:

- A arquitetura permite evolução do sistema?
- Componentes podem ser modificados isoladamente?
- Existem pontos de limitação (bottlenecks) no sistema?

Refinamento iterativo da arquitetura com IA



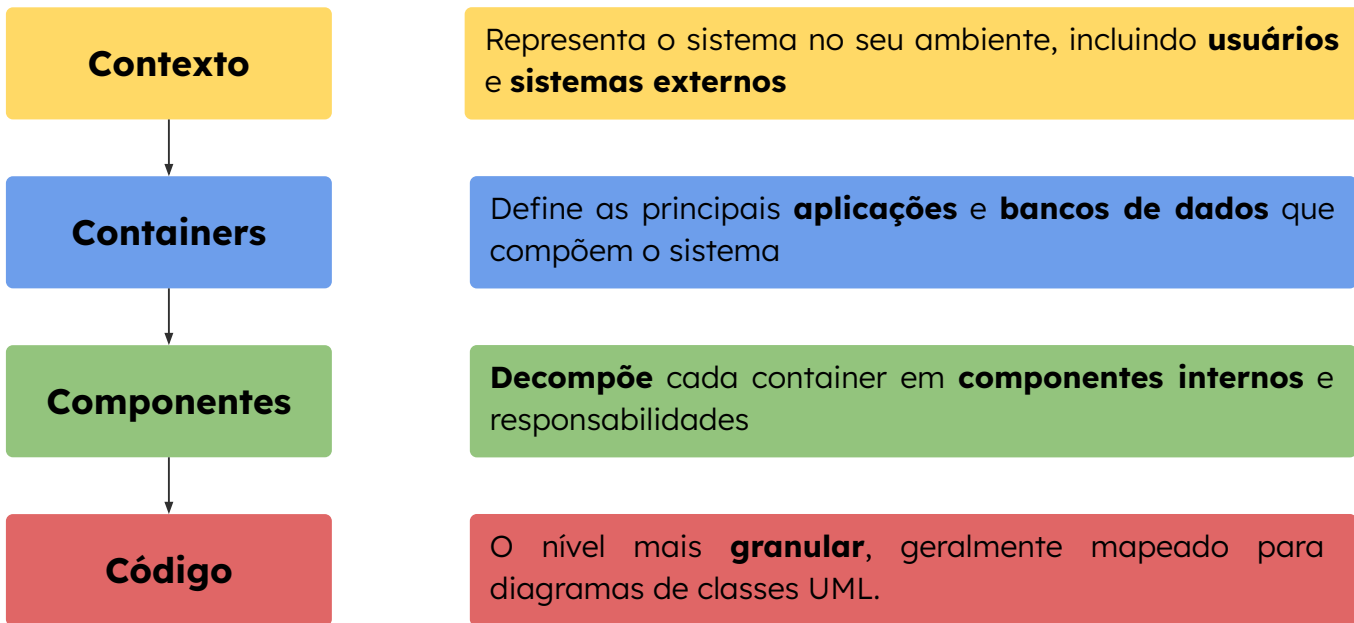
Iteração contínua

O refinamento envolve:

- Ajuste de responsabilidades
- Melhoria do fluxo
- Redução de acoplamento
- Aumento de clareza estrutural

Arquitetura com IA é um processo iterativo de melhoria contínua

Modelo C4 para representação de arquitetura

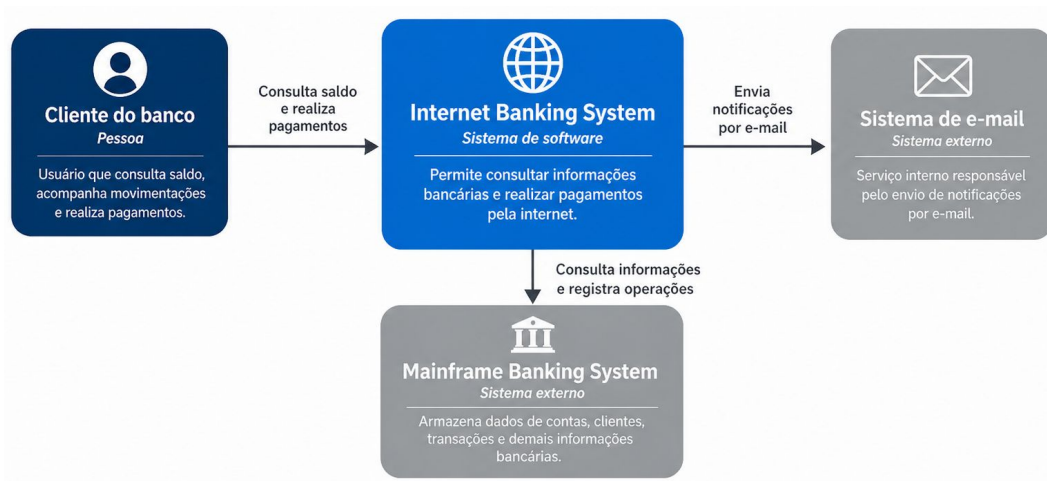


C4 permite navegar entre visão macro e detalhamento técnico

C4 — Nível de contexto

Objetivo do nível de contexto

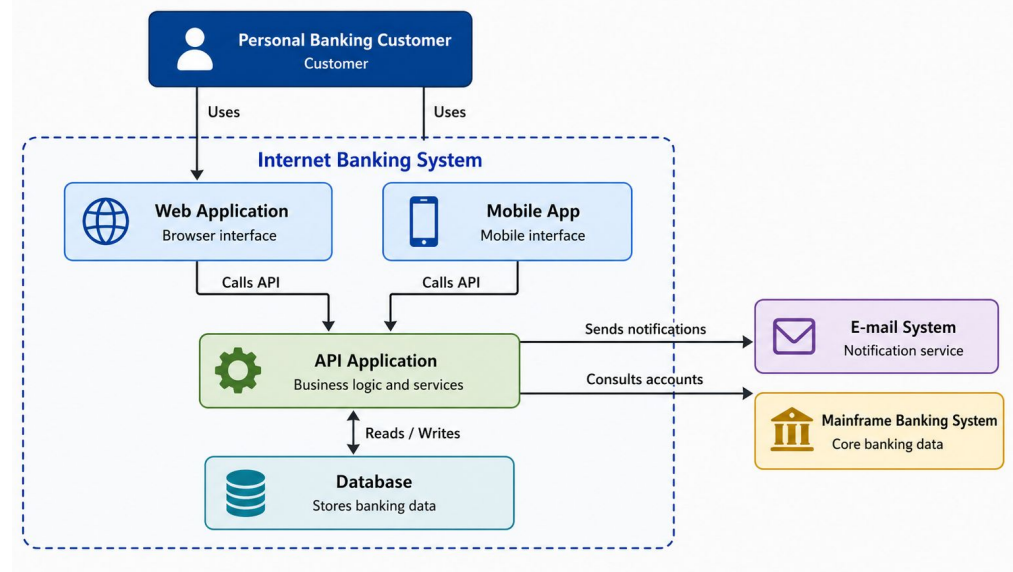
- Identificar o sistema principal
- Mostrar os atores que interagem com ele
- Evidenciar sistemas externos relacionados
- Representar interações em alto nível



C4 – Nível de containers

Objetivo do nível de containers

- Detalhar a estrutura principal do sistema
- Identificar aplicações, serviços e bancos de dados
- Mostrar como os containers se comunicam
- Explicitar responsabilidades dos principais blocos

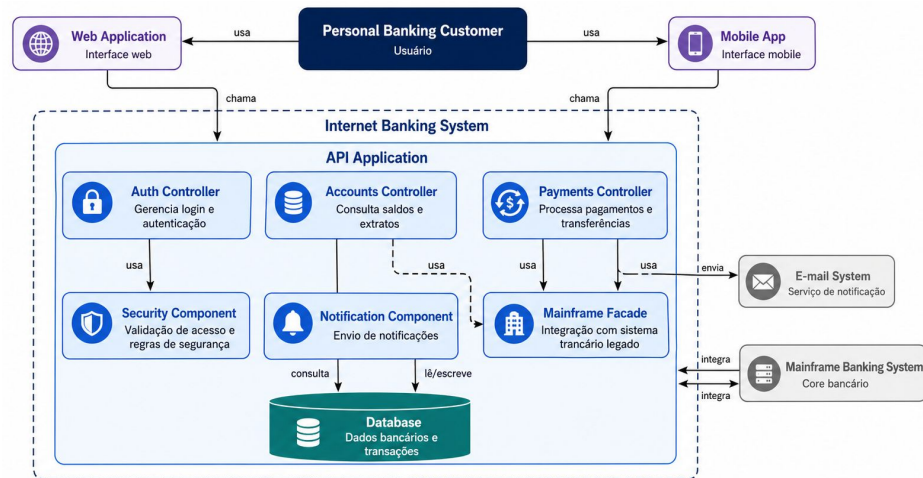


Containers mostram as principais unidades executáveis e armazenadoras do sistema.

C4 – Nível de componentes

Objetivo do nível de componentes

- Detalhar a estrutura interna de um container
- Identificar módulos, serviços e responsabilidades
- Mostrar como os componentes colaboram entre si
- Evidenciar onde decisões e regras são aplicadas



Componentes mostram como um container é dividido internamente em responsabilidades menores.

C4 — Nível de código

Objetivo do nível de código

- Representar como um componente é implementado internamente
- Mostrar pacotes, módulos, arquivos e organização interna
- Aproximar a arquitetura da estrutura real do projeto
- Apoiar entendimento técnico, evolução e manutenção

```
class TransactionRepository:  
    def __init__(self, ai_client, product_repository):  
        self.ai_client = ai_client  
        self.product_repository = product_repository  
  
    def transaction_repository(self, user_id):  
        products = self.product_repository.get_recent_products()  
        return self.ai_client.generate_recommendations(user_id, products)
```

API Application

```
└─ backend/  
    └─ controllers/  
        └─ auth_controller.py  
        └─ payments_controller.py  
    └─ services/  
        └─ account_service.py  
        └─ payment_service.py  
    └─ clients/  
        └─ mainframe_client.py  
        └─ email_client.py  
    └─ repositories/  
        └─ transaction_repository.py  
    └─ models/  
        └─ account.py  
        └─ transaction.py
```

Código mostra como os componentes são materializados na implementação.

UML como linguagem de modelagem visual

UML (*Unified Modeling Language*) é uma linguagem visual usada para modelar, documentar e comunicar sistemas de software.

Para que serve

- Documentar estrutura e comportamento do sistema
- Representar atores, entidades, fluxos e interações
- Apoiar comunicação entre áreas técnicas e de negócio
- Reduzir ambiguidades antes da implementação

Tipos mais usados na prática

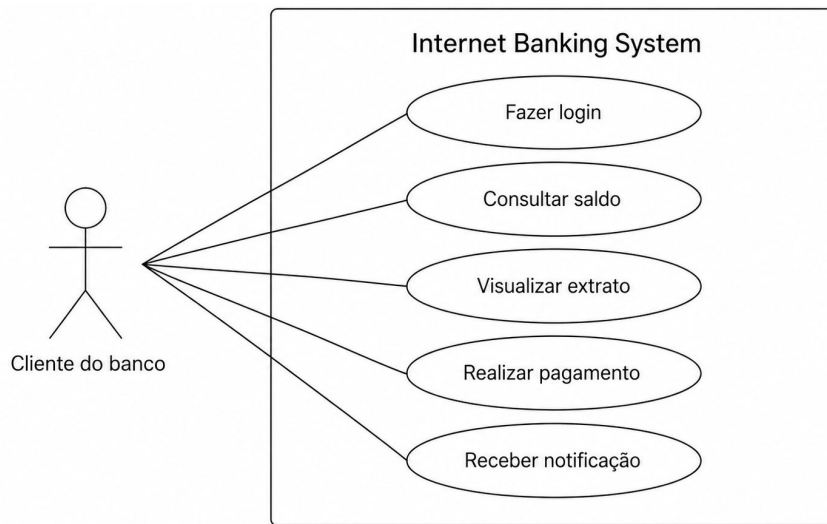
- Diagrama de caso de uso
- Diagrama de classes
- Diagrama de sequência
- Diagrama de atividades
- Diagrama de estados

C4 organiza a arquitetura em **níveis**. UML detalha **estrutura, comportamento e interações do sistema**.

UML – Diagrama de caso de uso

Representa a interação entre atores externos e funcionalidades do sistema.

- Identificar quem interage com o sistema
- Mapear funcionalidades disponíveis
- Delimitar o escopo funcional
- Apoiar a comunicação com áreas de negócio



O diagrama de caso de uso mostra quais funcionalidades do sistema são acessadas por cada ator

UML – Diagrama de classes

Representa classes, atributos, métodos e relacionamentos estruturais do sistema.

- Modelar entidades principais do domínio
- Representar atributos e comportamentos
- Identificar relacionamentos entre classes
- Apoiar a implementação orientada a objetos

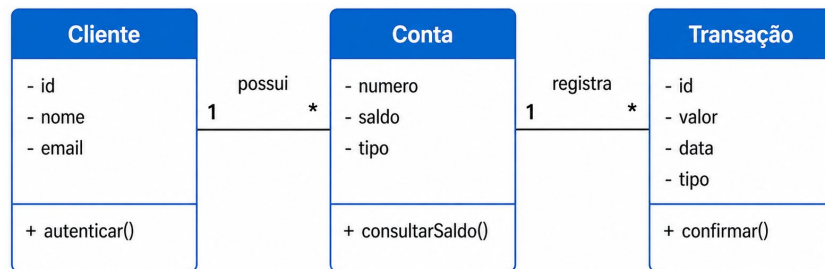


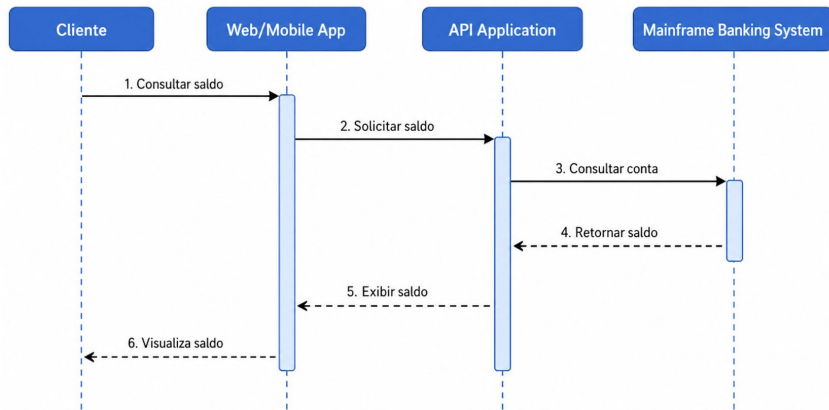
Diagrama de classes mostra a estrutura estática do domínio do sistema.

UML – Diagrama de sequência

Representa a troca de mensagens entre atores e componentes ao longo do tempo.

- Descrever a sequência de interações de uma funcionalidade
- Mostrar a ordem das chamadas entre componentes
- Evidenciar responsabilidades em cada etapa
- Validar integrações entre interface, backend e serviços externos

Diagrama de sequência mostra a ordem das interações entre componentes durante a execução de um caso de uso.



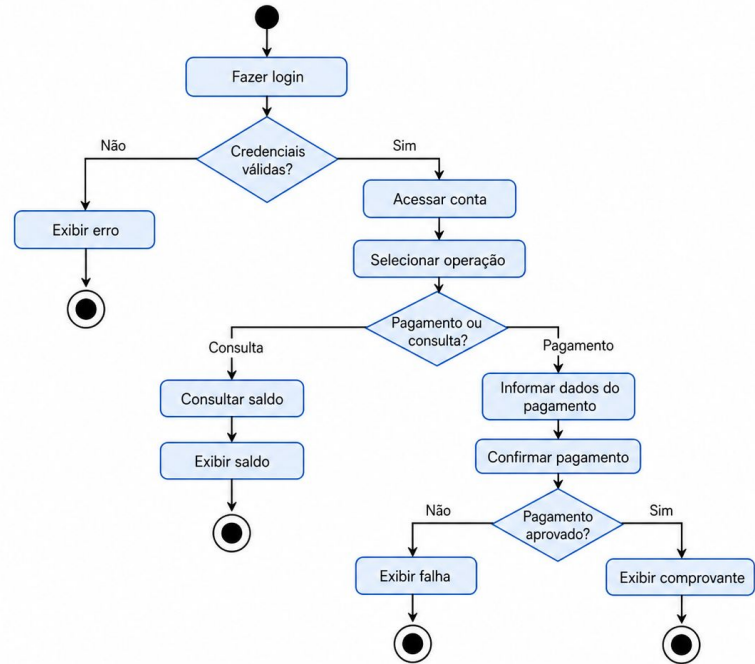
Exemplo de diagrama de sequência para consulta de saldo

UML – Diagrama de atividades

Representa o fluxo de atividades de um processo, incluindo decisões, desvios e caminhos alternativos.

- Modelar o fluxo de execução de um processo
- Representar decisões, desvios e ramificações
- Evidenciar início, término e caminhos alternativos
- Apoiar o entendimento de regras de negócio e navegação

O diagrama de atividades evidencia como um processo percorre etapas, decisões e resultados possíveis.

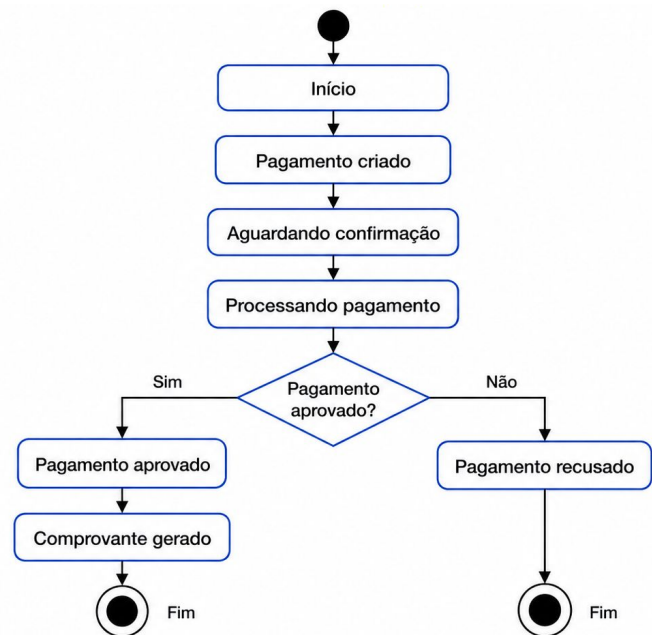


UML – Diagrama de estados

Representa os estados possíveis de uma entidade ao longo do seu ciclo de vida e as transições entre esses estados.

- Modelar o ciclo de vida de uma entidade
- Representar transições de estado ao longo do tempo
- Evidenciar eventos que disparam transições
- Apoiar a validação de regras de negócio e estados inválidos

Diagrama de estados mostra como uma entidade muda de situação durante seu ciclo de vida.



Estudo de caso: Shop4u

Contexto

- Shop4U
- App mobile de e-commerce com recomendação por IA

Qual é o sistema?

Requisitos

- Visualizar produtos
- Receber recomendações
- Adicionar ao carrinho
- Finalizar compra
- Receber confirmação

O que o sistema faz?

Elementos

- Cliente
- Mobile App
- Backend API
- Database
- Serviço de IA
- Gateway de Pagamento
- Serviço de Notificações

Quais partes existem?

Atividade prática: diagramas técnicos com IA

O que criar?	Ferramenta	Relações principais
• C4 Contexto • C4 Containers • Caso de uso • Sequência	FigJam com IA	• Cliente → Mobile App • Mobile App → Backend API • Backend API → Database • Backend API → Serviço de IA • Backend API → Gateway de Pagamento • Backend API → Serviço de Notificações



Manual de apoio

Da arquitetura para a experiência do usuário

Até aqui, modelamos o sistema do ponto de vista técnico:

- Arquitetura full-stack
- Backend, IA, APIs e banco de dados
- Representação C4
- Diagramas UML

Agora vamos olhar o mesmo sistema pela perspectiva do usuário:

- Como o usuário **navega**
- Como **percebe a interface**
- Como **executa suas tarefas**
- Como **validamos a experiência** antes da implementação

Arquitetura organiza o sistema; UX organiza a interação do usuário com esse sistema.

UX no contexto de produto

UX (User Experience) define como o usuário interage e percebe o sistema

- Fluxo de navegação entre funcionalidades
- Organização e hierarquia das interfaces
- Clareza das ações e feedback do sistema
- Eficiência e fluidez na execução de tarefas



UX traduz a arquitetura em experiência real para o usuário

Wireframe, Mockup e Protótipo

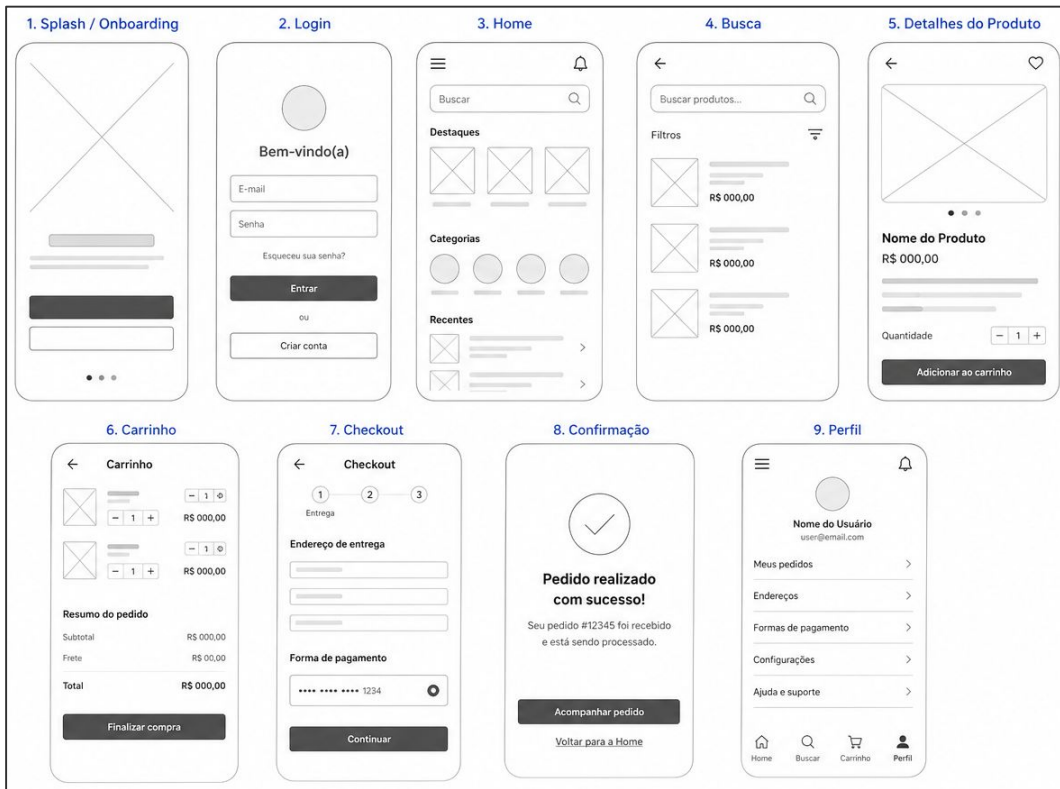
Wireframe	Mockup	Protótipo
• Estrutura da interface • Baixa fidelidade • Foco em layout e organização • Sem preocupação visual	• Representação visual da interface • Alta fidelidade • Cores, tipografia e estilo • Não interativo	• Simulação de interação • Navegável / clicável • Representa fluxo do usuário • Pode ser testado

Wireframe

- Estrutura da interface
- Baixa fidelidade
- Foco em layout, hierarquia e organização
- Sem definição de identidade visual

Objetivo

Validar estrutura, hierarquia e fluxo antes da definição visual da interface



Decisões de UX são tomadas antes do visual final

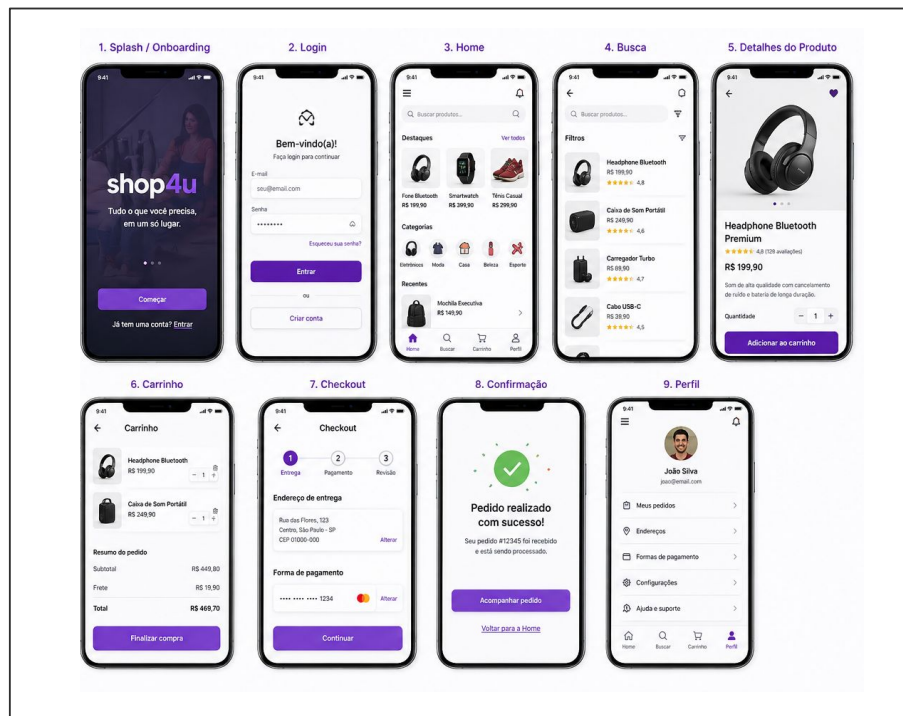
Mockup (alta fidelidade)

Alta fidelidade: aparência próxima do produto final, com cores, tipografia e identidade visual

- Representação visual da interface
- Alta fidelidade
- Uso de cores, tipografia e identidade visual
- Elementos próximos do produto final
- Não necessariamente interativo

Objetivo

Validar aparência, identidade visual e percepção do usuário



Mockup valida a aparência e a percepção visual do usuário

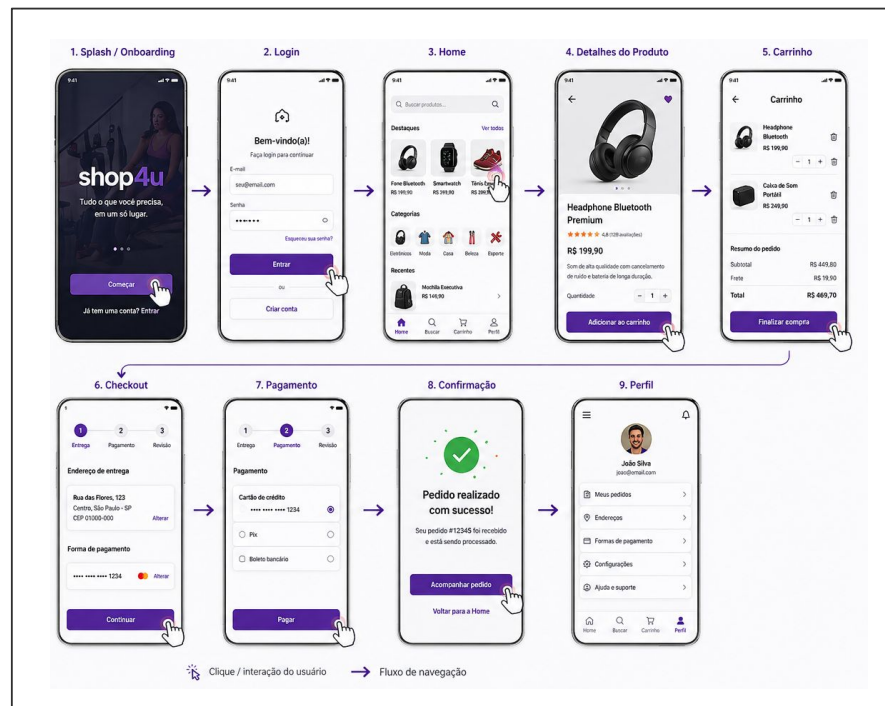
Protótipo (interação)

Simulação de interação do usuário com fluxo navegável entre telas

- Simulação da interação do usuário
- Fluxo navegável entre telas
- Representa o comportamento do sistema
- Pode ser clicável ou animado
- Permite validação com usuários

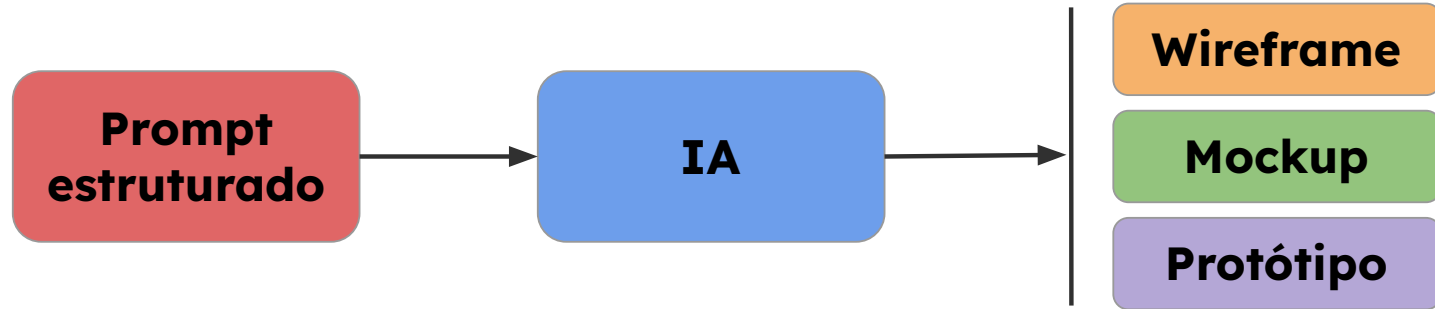
Objetivo

Validar fluxo de navegação, interação e usabilidade do sistema



Protótipo valida comportamento e usabilidade, não apenas aparência

Geração de UI com IA



IA reduz o tempo de criação, mas não substitui decisões de UX

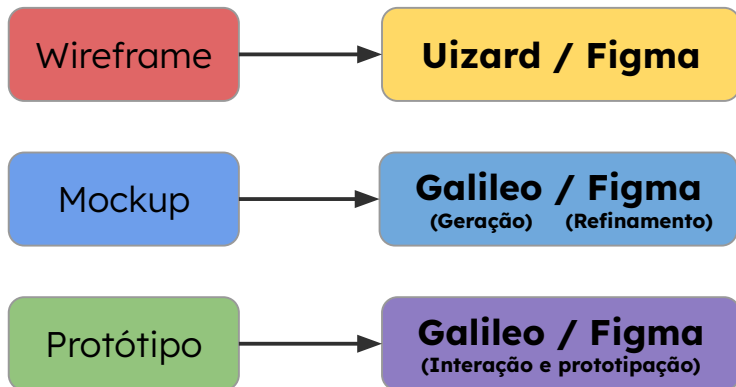
Ferramentas para geração de UI com IA

Figma (Make)	Uizard	Galileo AI
• Design colaborativo • Geração assistida de layouts • Integração com design system	• Geração rápida de wireframes e protótipos • Baseado em texto ou imagem • Foco em velocidade	• Geração de interfaces a partir de prompt • Foco em alta fidelidade • Design mais próximo do produto final

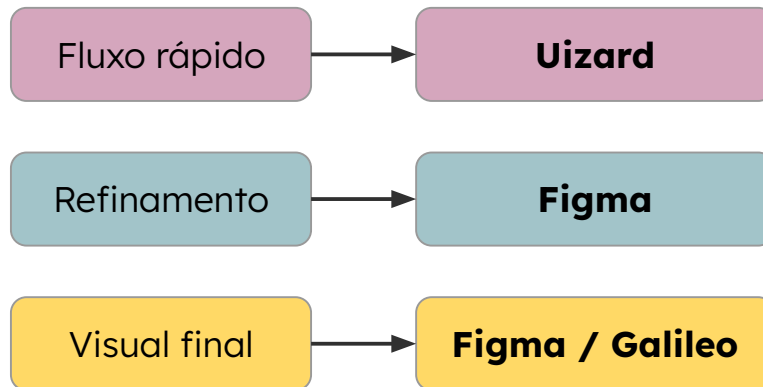
Cada ferramenta otimiza uma etapa diferente do processo de design

Quando usar cada ferramenta

Etapa do design



Contexto de uso



Ferramentas aceleram etapas específicas, mas não substituem o processo de design

Demonstração prática: geração de UI com IA

Vamos gerar uma interface com IA a partir de um prompt estruturado

Objetivo:

- Criar uma interface inicial (wireframe / mockup)
- Avaliar criticamente o resultado gerado
- Refinar a interface de forma iterativa com novos prompts

Etapas:

1. Definição do prompt
2. Geração da interface
3. Análise crítica do resultado
4. Refinamento iterativo

Análise dos resultados da geração de UI com IA

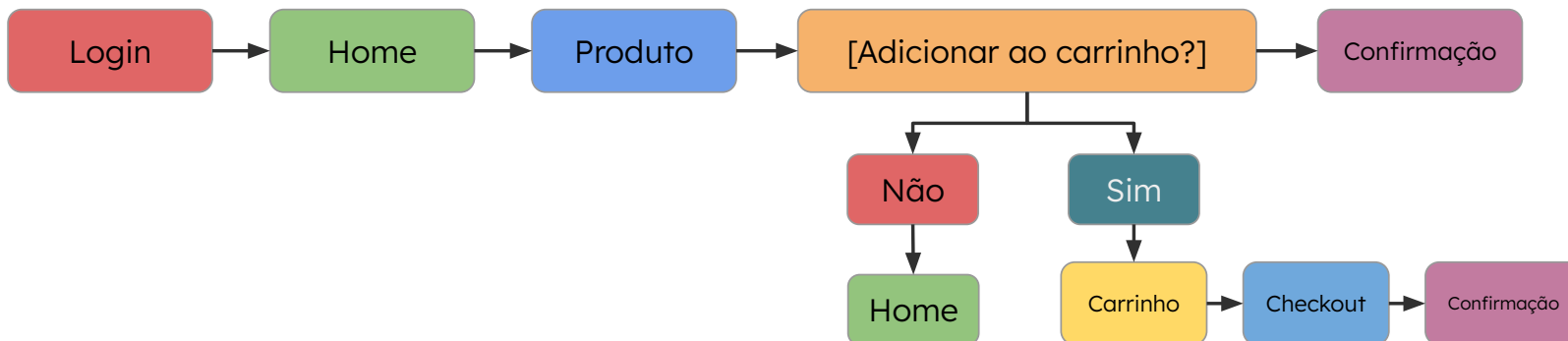
- A IA gera interfaces rapidamente a partir de um prompt
- O resultado inicial pode ser incompleto ou inconsistente
- A qualidade depende da clareza do prompt
- A análise crítica é essencial para validar a interface
- O refinamento iterativo melhora significativamente o resultado

Geração de fluxos de usuário (user flow) com IA

Fluxo simples



Fluxo com decisões



Conclusão Dia 1

Artefatos consolidados nesta etapa:

- Problema definido
- Arquitetura inicial do sistema
- Representações C4 e UML
- Wireframes, mockups e protótipo

Próxima etapa: transformar a modelagem em requisitos estruturados

- User stories
- Critérios de aceitação
- Backlog inicial
- Documentação inicial



Arquitetura e Design de Sistemas com IA

Criação de user stories, critérios de aceitação, backlog e documentação
inicial com IA

MÓDULO 1 SEMANA 05 - AULA 02



asCTI.SC

Recapitulação do Dia 1

No encontro anterior, estruturamos a visão inicial do sistema:

- Definição do problema e escopo inicial
- Arquitetura full-stack com IA
- Orquestração entre backend, IA, APIs e banco de dados
- Representações arquiteturais com C4 e UML
- Wireframes, mockups, protótipo e fluxo de navegação



Objetivo do Dia 2

Transformar a modelagem inicial em artefatos de desenvolvimento:

- Criar user stories a partir dos fluxos e funcionalidades mapeadas
- Definir critérios de aceitação claros e testáveis
- Estruturar um backlog inicial com apoio de IA
- Gerar documentação inicial com base nos artefatos definidos



Estruturação de requisitos a partir da modelagem

Os artefatos definidos até aqui ajudam a responder:

- Como o sistema será estruturado
- Quais componentes participam da solução
- Como o usuário navega pela interface
- Quais interações precisam existir

Agora precisamos transformar essa modelagem em requisitos:

- Claros
- Testáveis
- Priorizáveis
- Orientados a valor

Requisitos conectam a visão do sistema ao trabalho de desenvolvimento.

Definição de user story

User story é uma descrição funcional curta, escrita pela perspectiva do usuário, que representa uma necessidade e o valor esperado de uma funcionalidade.

Para que serve	Estrutura padrão
• Traduzir necessidades do usuário em requisitos implementáveis • Manter foco no valor entregue pela funcionalidade • Reduzir ambiguidade entre negócio, produto e desenvolvimento • Apoiar priorização, refinamento e validação	Como [tipo de usuário] , quero [ação ou funcionalidade] , para que [benefício ou objetivo] .

User stories conectam necessidade, funcionalidade e valor.

Estrutura da user story

Uma user story deve deixar claro quem precisa da funcionalidade, o que será feito e qual valor será entregue.

Como **[tipo de usuário]**

Define quem utiliza ou se beneficia da funcionalidade.

Exemplos:

- Cliente
- Administrador
- Operador de suporte

Quero **[ação ou funcionalidade]**

Define a ação esperada ou a funcionalidade que o sistema deve oferecer.

Exemplos:

- consultar saldo
- adicionar produto ao carrinho
- classificar uma solicitação

Para que **[benefício ou objetivo]**

Define o valor gerado pela funcionalidade.

Exemplos:

- acompanhar minha conta
- concluir uma compra
- direcionar o atendimento corretamente

Uma boa user story explicita usuário, funcionalidade e valor

Exemplo de user story

Uma user story descreve de forma curta quem precisa da funcionalidade, o que deseja fazer e qual valor espera obter

USER STORY

Como usuário,
Quero informar um problema com meu chip,
Para que o sistema possa identificar o tipo de falha e sugerir o encaminhamento adequado.

Quem

Usuário

Ação

Informar
problema com
o chip

Valor

Receber
encaminhamento
adequado

A user story conecta usuário, funcionalidade e valor de negócio

Avaliação de uma boa user story

Uma user story deve ser clara, orientada a valor e possível de validar.

User Story (Versão Inicial)

Como usuário,
quero acompanhar meu chamado,
para que eu saiba o que está acontecendo.

- Ação genérica
- Resultado esperado pouco específico
- Não define quais informações devem aparecer
- Dificulta a criação de critérios de aceitação

User Story (Versão Refinada)

Como usuário,
quero visualizar o status do meu chamado,
para que eu saiba se ele está em análise,
resolvido ou aguardando atendimento.

- Ação mais clara
- Resultado esperado específico
- Valor mais fácil de validar
- Facilita a definição dos critérios de aceitação

User stories vagas geram implementação ambígua e validação fraca

Critérios de aceitação

Critérios de aceitação definem as condições que precisam ser atendidas para considerar uma user story concluída.

User Story	Critérios de aceitação
Como usuário, quero visualizar o status do meu chamado, para acompanhar se ele está em análise, aguardando atendimento ou resolvido.	• O sistema deve exibir o status atual do chamado. • O status deve estar visível na tela de detalhes do chamado. • O sistema deve suportar os seguintes status: Em análise, Aguardando atendimento e Resolvido. • Se o chamado não for encontrado, o sistema deve exibir uma mensagem de erro apropriada.

Critérios de aceitação tornam a user story testável

Critérios de aceitação em BDD

BDD organiza critérios de aceitação em cenários verificáveis, descrevendo contexto, ação executada e resultado esperado.

Dado que / *Given* [contexto inicial]

Quando / *When* [ação executada]

Então / *Then* [resultado esperado]

Dado que o usuário possui um chamado cadastrado,
Quando ele acessa a tela de detalhes do chamado,
Então o sistema deve exibir o status atual do chamado.

BDD não substitui a user story. Ele transforma os critérios de aceitação em cenários verificáveis

Exemplo de critérios de aceitação em BDD

Uma mesma user story pode gerar múltiplos cenários de validação.

User story de referência

Como usuário,
quero visualizar o status do meu chamado,
para acompanhar se ele está em análise,
aguardando atendimento ou resolvido.

Cenário 1 Fluxo principal

Dado que o usuário possui um chamado cadastrado,
Quando ele acessa a tela de detalhes do chamado,
Então o sistema deve exibir o status atual do chamado.

Cenário 2 Exceção: chamado não encontrado

Dado que o usuário acessa um chamado inexistente,
Quando o sistema tenta carregar os detalhes do chamado,
Então deve exibir uma mensagem informando que o chamado não foi encontrado.

Cenário 3 Variação de *status*

Dado que o chamado está aguardando atendimento,
Quando o usuário acessa os detalhes do chamado,
Então o status exibido deve ser “Aguardando atendimento”.

Critérios em BDD validam o comportamento esperado em diferentes cenários de uso.

Problemas comuns em requisitos gerados por IA

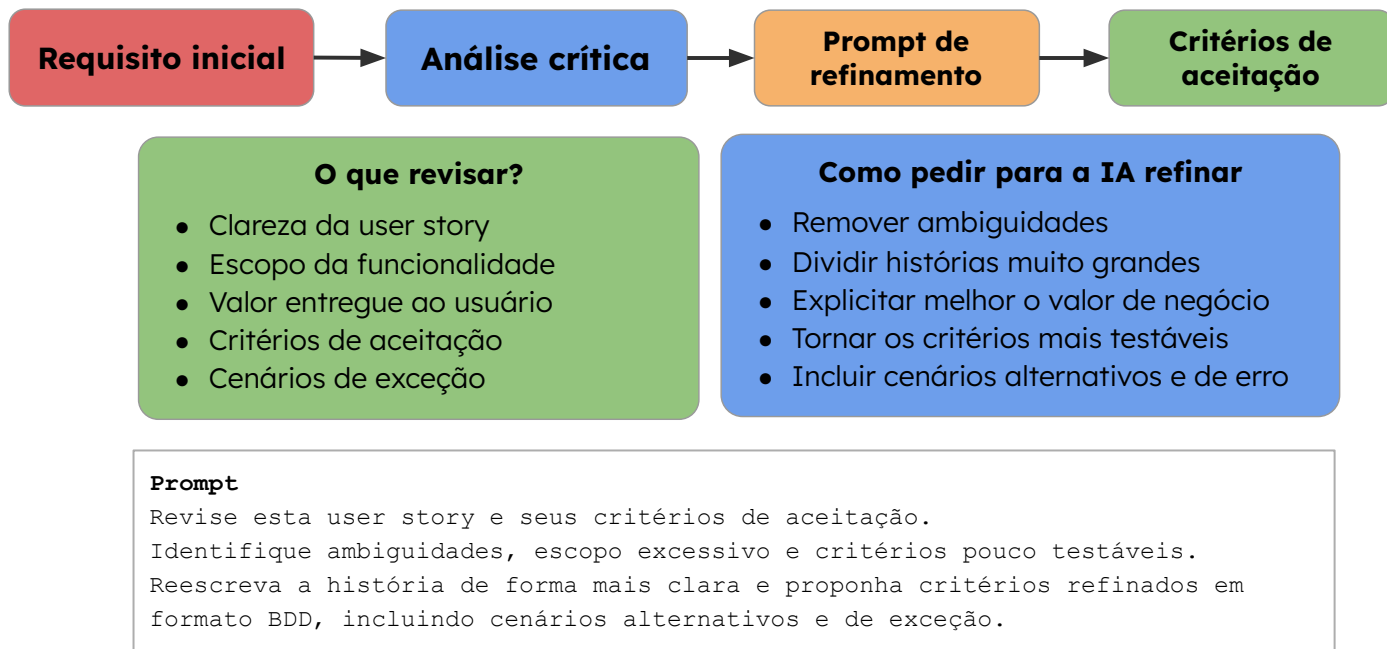
A IA pode acelerar a criação de user stories e critérios, mas os requisitos gerados ainda precisam de revisão técnica e validação humana.

Ambiguidade	Escopo amplo	Valor pouco explícito	Baixa testabilidade
• Termos genéricos ou pouco verificáveis • Falta de clareza sobre comportamento esperado	• User stories grandes demais • Muitas funcionalidades em uma única história	• Benefício fraco ou ausente • Foco excessivo na tarefa, não no resultado	• Condições subjetivas • Falta de cenários de exceção

Requisitos gerados por IA devem ser tratados como rascunhos estruturados, não como especificações finais

Refinamento de requisitos com IA

Após a geração inicial, a IA pode apoiar a revisão, melhoria e detalhamento dos requisitos.



A IA acelera o refinamento dos requisitos, mas a validação final continua sendo responsabilidade humana.

O que é backlog?

Backlog é uma lista organizada e priorizada de itens que representam o trabalho necessário para evoluir um produto ou sistema.

Um backlog pode conter

- Épicos
- Features
- User stories
- Tarefas técnicas
- Bugs ou melhorias

Para que serve?

- Organizar o trabalho de desenvolvimento
- Priorizar o que deve ser entregue primeiro
- Dar visibilidade ao escopo e às prioridades do produto
- Conectar requisitos, implementação e entrega

(EXEMPLO) Backlog do sistema de suporte com IA

- **Épico:** Atendimento com IA
- **Feature:** Gestão de chamados
- **User story:** Como usuário, quero informar um problema com chip para receber o encaminhamento adequado
- **Tarefa técnica:** Implementar endpoint de classificação
- **Critério de aceitação:** O sistema deve classificar a falha e sugerir o encaminhamento correspondente

O backlog organiza e prioriza os itens que orientam a evolução do produto

Geração de backlog inicial com IA

A IA organiza artefatos definidos em backlog inicial, com épicos, features, user stories e tarefas técnicas.

Entradas

- Problema definido
- Fluxo do usuário
- User stories
- Critérios de aceitação
- Regras de negócio

Prompt

Organize os requisitos abaixo em um backlog inicial.

Considere:

- épicos
- features
- user stories
- tarefas técnicas
- prioridade inicial

Use uma estrutura clara e indique possíveis dependências entre os itens.

Saída esperada da IA

Épico: Atendimento com IA

Feature: Registro de chamado

- **User story:** Informar problema com chip
- **Tarefa técnica:** Criar endpoint de registro
- **Prioridade:** Alta

Feature: Classificação automática

- **User story:** Classificar problema informado
- **Tarefa técnica:** Integrar modelo de IA
- **Prioridade:** Alta

Feature: Acompanhamento de chamado

- **User story:** Visualizar status do chamado
- **Tarefa técnica:** Criar tela de detalhes
- **Prioridade:** Média

Revisão humana

- Escopo dos itens
- Ordem de prioridade
- Dependências técnicas
- Clareza das user stories
- Viabilidade de implementação

Priorização do backlog com IA

Backlog inicial → Critérios de priorização → Sugestão da IA → Revisão do time

Critérios de priorização

- Valor para o usuário
- Impacto no negócio
- Urgência
- Dependências técnicas
- Esforço estimado
- Risco de implementação

Prompt

Analise o backlog abaixo e sugira uma priorização inicial.

Considere:

- valor para o usuário
- impacto no negócio
- dependências técnicas
- esforço estimado
- riscos de implementação

Classifique os itens como Alta, Média ou Baixa prioridade e justifique cada decisão.

Saída esperada da IA

Alta prioridade

- Registro de chamado

Justificativa: é necessário para iniciar o fluxo principal do sistema.

Alta prioridade

- Classificação automática

Justificativa: é a funcionalidade central do sistema com IA.

Média prioridade

- Acompanhamento de chamado

Justificativa: agrega valor ao usuário, mas depende do registro do chamado.

Documentação inicial com IA

Entradas

- Arquitetura inicial
- Diagramas C4 / UML
- User stories
- Critérios de aceitação
- Backlog priorizado

Prompt

Com base nos artefatos abaixo, gere uma documentação inicial do sistema.

Inclua:

- visão geral da solução
- principais funcionalidades
- arquitetura resumida
- backlog inicial
- critérios de aceitação
- pontos de atenção e dependências

Saída esperada da IA

Documentação inicial:

- Visão geral do sistema
- Objetivo da solução
- Escopo funcional
- Arquitetura resumida
- User stories principais
- Critérios de aceitação
- Dependências e riscos

Revisão humana

Validar:

- Coerência técnica
- Escopo descrito
- Decisões arquiteturais
- Consistência dos requisitos
- Lacunas ou informações ausentes

Ferramentas para documentação com IA

Ferramenta	Indicada para
Confluence + Roovo	Documentação corporativa, páginas de projeto, requisitos, decisões e conhecimento interno
Notion AI	Documentação leve, wikis, PRDs, organização de notas, tarefas e conteúdos colaborativos
ChatGPT + Markdown	Geração controlada de documentação a partir de artefatos fornecidos no prompt
GitHub Copilot	README, documentação próxima ao código, explicação de módulos e atualização de docs no repositório
Mintlify	Documentação técnica para desenvolvedores, APIs, docs-as-code e portais de documentação

Exemplo de documentação gerada

Sistema de Suporte com IA

1. Visão geral

Sistema para receber solicitações de usuários, interpretar o problema informado e sugerir encaminhamentos automáticos.

2. Objetivo

Apoiar o atendimento inicial por meio de classificação automática, priorização e sugestão de ação.

3. Escopo funcional

- Registrar solicitação do usuário
- Classificar o tipo de problema
- Sugerir encaminhamento adequado
- Exibir status do chamado

4. Arquitetura resumida

A solução é composta por frontend, backend, serviço de IA, APIs externas e banco de dados.

Exemplo de documentação gerada

5. Requisitos principais

- O usuário deve informar um problema com o chip
- O sistema deve classificar a solicitação
- O sistema deve exibir o status do chamado

6. Critérios de aceitação

- O status atual do chamado deve ser exibido na tela de detalhes
- Chamados inexistentes devem retornar mensagem de erro
- A classificação deve sugerir um encaminhamento correspondente

7. Pontos de atenção

- Revisar limites da classificação automática
- Validar cenários de erro
- Definir responsáveis pela priorização final

Conclusão Dia 2

Artefatos consolidados nesta etapa:

- Requisitos funcionais estruturados em user stories
- Critérios de aceitação verificáveis
- Cenários BDD para validação funcional
- Backlog inicial priorizado
- Documentação inicial do sistema

Próxima etapa: transformar a modelagem em requisitos estruturados

- Organização do repositório
- Estratégia de branches
- Fluxo de Pull Requests
- Convenções de código e documentação
- Uso da IA para apoiar padronização e revisão



Arquitetura e Design de Sistemas com IA

Organização de repositório, criação de branches, PRs e convenções
geradas por IA

MÓDULO 1 SEMANA 05 - AULA 03



asCTI.SC

Recapitulação do Dia 2

No encontro anterior, transformamos a modelagem inicial em artefatos de desenvolvimento:

- User stories orientadas a valor
- Critérios de aceitação claros e testáveis
- Cenários em BDD para validação
- Backlog inicial organizado e priorizado
- Documentação inicial do sistema



Objetivo do Dia 3

Ao final do encontro, o aluno deverá ser capaz de:

- Estruturar um repositório de projeto
- Definir convenções de branches e commits
- Criar Pull Requests com template
- Organizar documentação e evidências de prompts
- Usar IA para apoiar padronização, revisão e organização do fluxo



Alinhamento com o mini-projeto

Artefatos exigidos	Evidências no repositório
• README.md • docs/PRD.md • prompts.md • Pull Request • Commits	• Documentação do projeto • Documento de produto • Uso da IA • Fluxo de revisão • Participação do grupo

Fluxo de desenvolvimento orientado pelo backlog

Backlog é a lista priorizada de itens que representam o trabalho a ser desenvolvido no projeto.

Rastreabilidade do artefato



Sequência de execução

1. Item priorizado no backlog
2. Transformação em tarefa de desenvolvimento
3. Criação da branch
4. Registro incremental em commits
5. Abertura do Pull Request
6. Integração ao branch principal

O backlog orienta a execução; o repositório registra a implementação.

O que é um repositório de software?

Repositório é o espaço onde o projeto é versionado, organizado e compartilhado pelo time.

Um repositório pode conter:

- Código-fonte da aplicação
- Documentação técnica e funcional
- Arquivos de configuração
- Testes automatizados
- Histórico de commits
- Branches e Pull Requests
- Evidências do uso de IA no processo

```
repository/  
├── src/  
├── tests/  
├── docs/  
├── prompts.md  
├── README.md  
└── .github/
```

Estrutura inicial de um repositório

Elemento	Função
src/	Código-fonte principal da aplicação
tests/	Testes automatizados do projeto
docs/	Documentação técnica e materiais de apoio
README.md	Visão geral, instruções de uso e contexto
requirements.txt ou package.json	Lista de dependências do projeto
.gitignore	Arquivos e pastas que não serão versionados

Criação do repositório com apoio da IA

A IA pode acelerar a criação da estrutura inicial do projeto.

- **Entrada:** tipo de aplicação, stack, requisitos e backlog
- **Prompt:** solicitação clara da estrutura esperada
- **Saída:** estrutura de pastas, arquivos base e documentação inicial
- **Revisão:** validação técnica pela equipe

Exemplo de Prompt:

Crie uma estrutura inicial de repositório para uma aplicação web com backend em Python e frontend em React.

Considere:

- separação entre frontend e backend
- pasta para testes automatizados
- arquivo README.md
- controle de dependências
- boas práticas de organização

Exemplo de prompt para criação de repositório

Você é um arquiteto de software responsável por propor a organização inicial de um projeto.

Objetivo:

Criar uma estrutura inicial de repositório para uma aplicação web com backend em Python e frontend em React.

Contexto:

O projeto terá backend, frontend, testes automatizados e documentação técnica.

Regras:

- Separar claramente backend e frontend.
- Incluir pastas para testes.
- Incluir arquivo README.md.
- Incluir arquivos de dependências.
- Evitar uma estrutura complexa demais para um projeto inicial.

Formato de saída:

Retorne a estrutura em formato de pastas e arquivos e explique brevemente a função de cada item.

Proposta de estrutura gerada pela IA

```
meu-projeto/  
  backend/  
    src/  
    tests/  
    requirements.txt
```

```
frontend/  
  src/  
  tests/  
  package.json
```

```
docs/  
  arquitetura.md  
  requisitos.md
```

```
README.md  
.gitignore
```

Validar antes de aplicar

- Adequação à stack
- Separação entre backend e frontend
- Presença de testes e documentação
- Simplicidade da estrutura
- Arquivos essenciais

IA como agente executando ações

Exemplo de comandos que um agente poderia executar:

```
mkdir -p meu-projeto/backend/src
mkdir -p meu-projeto/backend/tests
mkdir -p meu-projeto/frontend/src
mkdir -p meu-projeto/frontend/tests
mkdir -p meu-projeto/docs
touch meu-projeto/README.md
touch meu-projeto/.gitignore
touch meu-projeto/backend/requirements.txt
touch meu-projeto/frontend/package.json
```

Para executar ações, a IA precisa de ferramentas, como:

- terminal
- GitHub
- VS Code/Copilot
- Cursor
- agente com acesso ao sistema de arquivos
- pipeline automatizado
- ferramenta de criação de projetos

Versionamento com Git

Git é uma ferramenta de controle de versão que registra as alterações feitas no projeto ao longo do tempo.

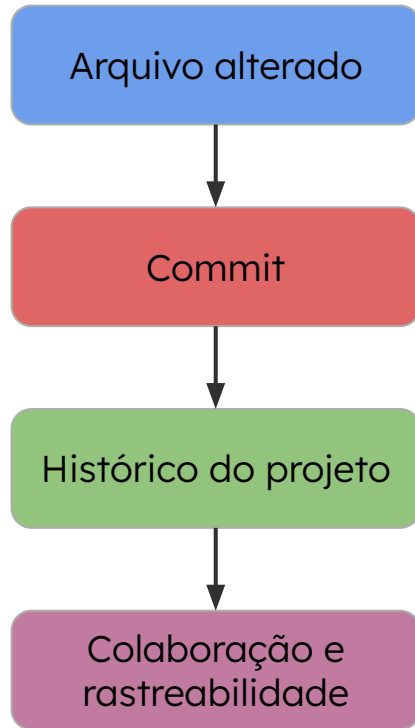


O que mudou?

Quando mudou?

Por que mudou?

Visão geral do processo de versionamento



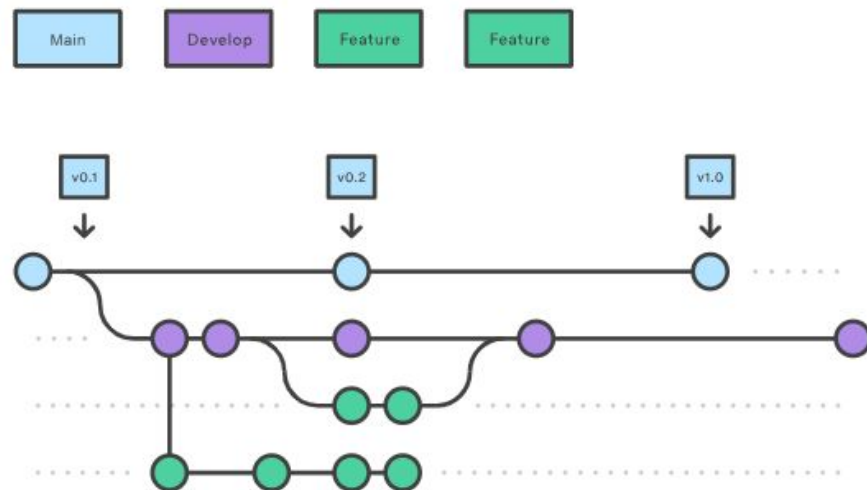
Conceito	Papel no desenvolvimento
Arquivo alterado	Código, documentação ou configuração modificada
Commit	Registro de uma alteração relevante no projeto
Histórico	Linha do tempo das mudanças realizadas
Reversão	Possibilidade de recuperar versões anteriores
Colaboração	Permite que várias pessoas trabalhem no mesmo projeto

Trabalhando com branches

Uma branch é uma linha paralela de desenvolvimento criada para modificar o projeto sem impactar diretamente a versão principal.

Por que usar branches?

- Isolar mudanças
- Reduzir risco
- Facilitar revisão
- Organizar entregas



Branches permitem trabalhar em paralelo sem comprometer a versão principal do projeto

Padrões de organização de branches

Git Flow

- main
- develop
- feature/*
- release/*
- hotfix/*

Projetos com versões planejadas

GitHub Flow

- main
- feature/*
- Pull Request
- merge

Entrega contínua e times ágeis

GitLab Flow

- main
- staging / pre-production
- production

Projetos com ambientes separados

Trunk-Based

- main
- short-lived branches
- feature flags

Times com CI/CD bem estabelecido

Termos comuns em estratégias de branching

Termo	Papel no fluxo
main	Linha principal do projeto
develop	Integração das funcionalidades em desenvolvimento
feature/*	Desenvolvimento de nova funcionalidade
release/*	Preparação de uma versão
hotfix/*	Correção urgente em produção
staging / pre-production	Validação antes da produção
production	Referência do ambiente produtivo
short-lived branches	Branches de curta duração
feature flags	Controle de ativação de funcionalidades

Nomeação de branches no projeto

Estrutura: tipo/descricao-curta

Tipo	Uso	Exemplo
feature/	Nova funcionalidade	feature/login
fix/	Correção de erro	fix/validacao-cadastro
docs/	Documentação	docs/atualizar-readme
refactor/	Melhoria interna	refactor/organizar-camadas
test/	Testes	test/adicionar-testes-login

O nome da branch ajuda o time a entender rapidamente a intenção da mudança

Commits no Git

Um commit registra uma alteração no projeto e cria um ponto no histórico.

Branch com commits

feature/login

- |— cria estrutura da tela
- |— adiciona validação de e-mail
- |— conecta formulário à API

Boas práticas

Um bom commit deve ser:

- pequeno
- claro
- relacionado
- rastreável
- revisável

Mensagens de commit

A mensagem de commit deve descrever a mudança de forma clara, curta e rastreável.

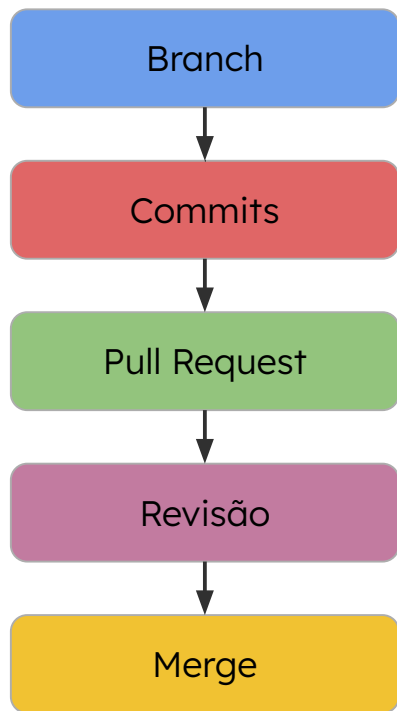
Estrutura: tipo: descrição curta da alteração

Tipo	Exemplo
feat	feat: adiciona tela de login
fix	fix: corrige validação de e-mail
docs	docs: atualiza instruções do README
refactor	refactor: reorganiza serviços de autenticação
test	test: adiciona testes para login

A mensagem deve explicar a intenção da mudança, não apenas indicar que algo foi alterado

Pull Request

É uma solicitação para revisar alterações antes de integrá-las ao branch principal do projeto



O que revisar em um PR?

- Escopo da alteração
- Código modificado
- Testes criados ou atualizados
- Documentação impactada
- Relação com a tarefa do backlog

O PR transforma uma alteração individual em uma decisão revisada pelo time.

Formas de revisar Pull Requests com IA

Manual com IA conversacional

Como funciona

Usuário copia o contexto do PR, envia descrição, tarefa e diff, e solicita uma análise.

Exemplo

LLM conversacional (ChatGPT, Claude, Gemini).

Vantagem

Simples para iniciar, sem integração com o repositório

Limite

Depende da qualidade do contexto fornecido.

Assistido na IDE

Como funciona

Ferramenta usa o contexto local do projeto: arquivos abertos, branch atual, dependências e código relacionado.

Exemplo

Assistente de código integrado à IDE (GitHub Copilot, Cursor, extensões no VSCode)

Vantagem

Maior contexto técnico durante o desenvolvimento

Limite

A análise depende do escopo acessível pela IDE e da interação do desenvolvedor

Automatizado no repositório

Como funciona

Bot ou workflow analisa o PR, gera comentários automaticamente e registra a revisão no GitHub/GitLab.

Exemplo

GitHub Actions, bots de revisão, integração com LLM

Vantagem

Padroniza a revisão e reduz esforço manual recorrente

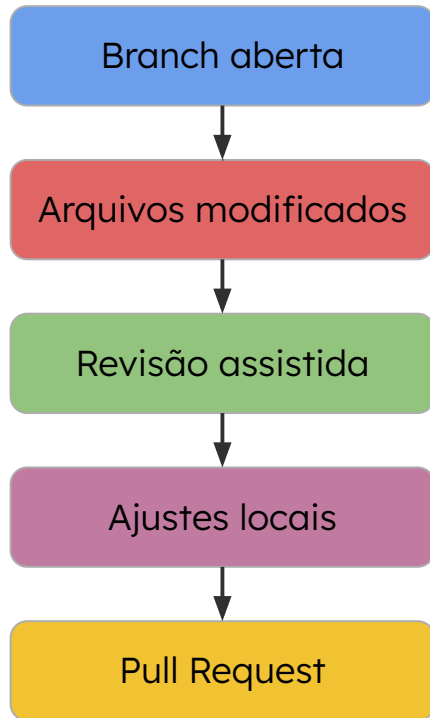
Limite

Exige configuração, permissões e critérios claros de revisão

Mais integração reduz o esforço manual, mas aumenta a complexidade de implementação.

Revisão assistida com IA na IDE

Fluxo

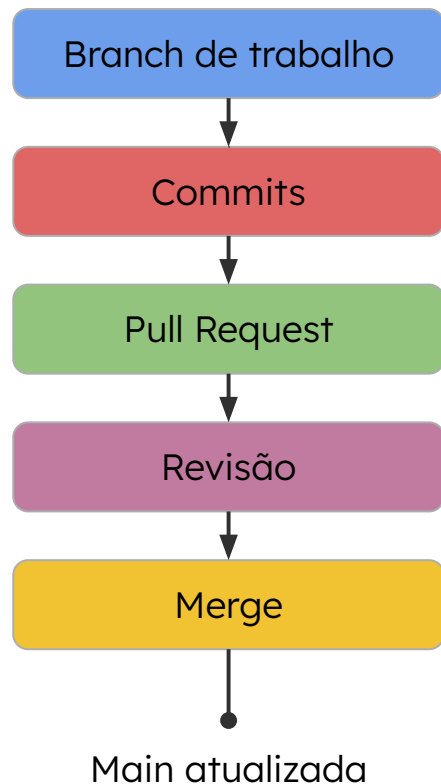


Crítérios de revisão

Revise as alterações locais da branch considerando:

- Clareza e organização do código
- Possíveis bugs ou inconsistências
- Aderência à tarefa do backlog
- Necessidade de testes
- Impacto na documentação
- Oportunidades de simplificação

Merge e integração ao branch principal



Antes do merge, validar:

- Pull Request revisado
- Testes executados
- Conflitos resolvidos
- Escopo validado
- Documentação atualizada, se necessário

O merge integra ao branch principal as alterações desenvolvidas, revisadas e aprovadas.

Fluxo completo: do backlog ao merge



Backlog indica a origem da demanda

Branch organiza a unidade de trabalho

Commits registram a evolução da implementação

Pull Request concentra revisão e discussão

Merge integra a alteração validada

Convenções do projeto

Convenções são acordos técnicos que definem como o time organiza, registra e revisa o trabalho no projeto.

Repositório

Código, testes, documentação e configs

Branches

Padrão de nomes e tipos de alteração

Commits

Mensagens claras e rastreáveis

Pull Request

Descrição, escopo e critérios de revisão

Merge

Critérios de validação antes da integração

Documentação

Manutenção de README, decisões técnicas e instruções de execução

Documento de convenções com apoio da IA



Convenções definidas

- Repositório
- Branches
- Commits
- Pull Request
- Merge
- Documentação

IA estrutura o conteúdo

- Organiza seções
- Padroniza linguagem
- Transforma acordos em instruções
- Gera versão inicial revisável

Saída esperada

- Fluxo de contribuição
- Padrões de branches
- Mensagens de commit
- Critérios de PR
- Checklist antes do merge

Exemplo prático de geração do CONTRIBUTING.md

Modo tradicional

1. Preparar contexto

Arquivo ou texto com as convenções definidas

2. Enviar prompt estruturado

Papel, objetivo, contexto, regras e formato

3. Copiar a saída

Criar CONTRIBUTING.md manualmente

Assistente na IDE

1. Criar arquivo de contexto

docs/convencoes_projeto.md

2. Solicitar geração na IDE

Assistente acessa o contexto do projeto

3. Criar arquivo no repositório

CONTRIBUTING.md na raiz

A diferença está em como o contexto chega à IA e onde a saída é aplicada.

Validação dos documentos gerados

README.md

Documentação inicial do projeto

- Objetivo do projeto
- Estrutura do repositório
- Instruções de uso
- Coerência com o escopo do projeto

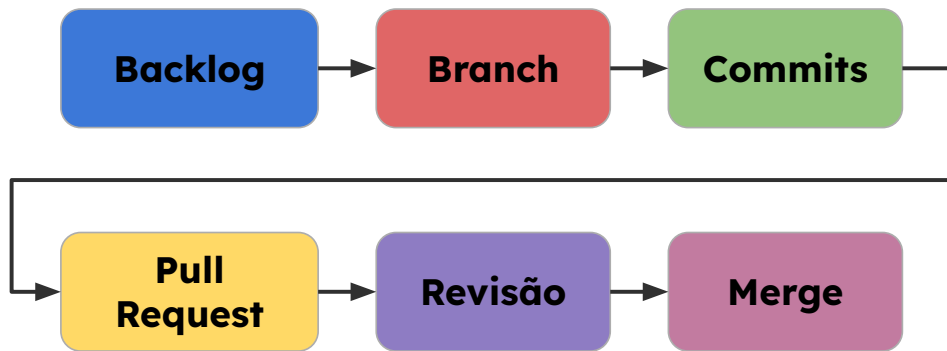
CONTRIBUTING.md

Convenções técnicas do projeto

- Convenções de branches
- Padrão de commits
- Critérios de Pull Request
- Checklist antes do merge
- Regras aplicáveis ao projeto

A IA organiza o texto; o time valida o conteúdo

Aplicação no mini-projeto



Documentos de apoio

- **README.md:**
visão geral e instruções do projeto
- **CONTRIBUTING.md:**
convenções de colaboração
- **docs/convencoes_projeto.md**
contexto técnico usado pela IA

O mini-projeto deve registrar a trajetória da mudança: origem, implementação, revisão e integração

Arquitetura e Desenvolvimento com IA

Dia 1 — Modelagem e experiência do usuário

- Definição do problema e papel da IA no sistema
- Arquitetura full-stack, C4 e diagramas UML
- Wireframes, mockups, protótipos e user flows

Dia 2 — Requisitos e backlog

- User stories orientadas a valor
- Critérios de aceitação e cenários em BDD
- Backlog priorizado e documentação inicial com IA

Dia 3 — Repositório e fluxo de desenvolvimento

- Estrutura inicial do repositório e convenções do projeto
- Branches, commits, Pull Requests, revisão e merge
- README.md, CONTRIBUTING.md e documentação assistida por IA



SCTEC

PREPARANDO
VOCÊ PARA O
AGORA_



SCITI.SC